

Deliverable D2

TrentoParking

David Dorobantu – 234467
Riccardo Gonzato – 246476
Matteo Sepa – 243283

1 Diagramma dei componenti

In questa sezione viene illustrata l'architettura logica di TrentoParking. Il sistema è stato progettato seguendo un approccio modulare, con l'obiettivo di separare chiaramente interfaccia utente, logica applicativa, persistenza dei dati e integrazione con servizi esterni. Questa scelta consente di ottenere un sistema più semplice da comprendere, aggiornare e mantenere nel tempo.

L'architettura adottata distingue:

- un **Web Frontend**, usato come punto di accesso dell'utente;
- un **Backend REST API**, che espone i servizi applicativi;
- una serie di **moduli di business logic**, responsabili delle funzionalità principali;
- un componente di **Persistenza Dati**;
- alcuni **sistemi esterni**, necessari per mappe, dati parcheggi, trasporto pubblico e pagamento mock.

1.1 Diagramma dei componenti

La Figura 1 riporta la panoramica dei componenti del sistema.

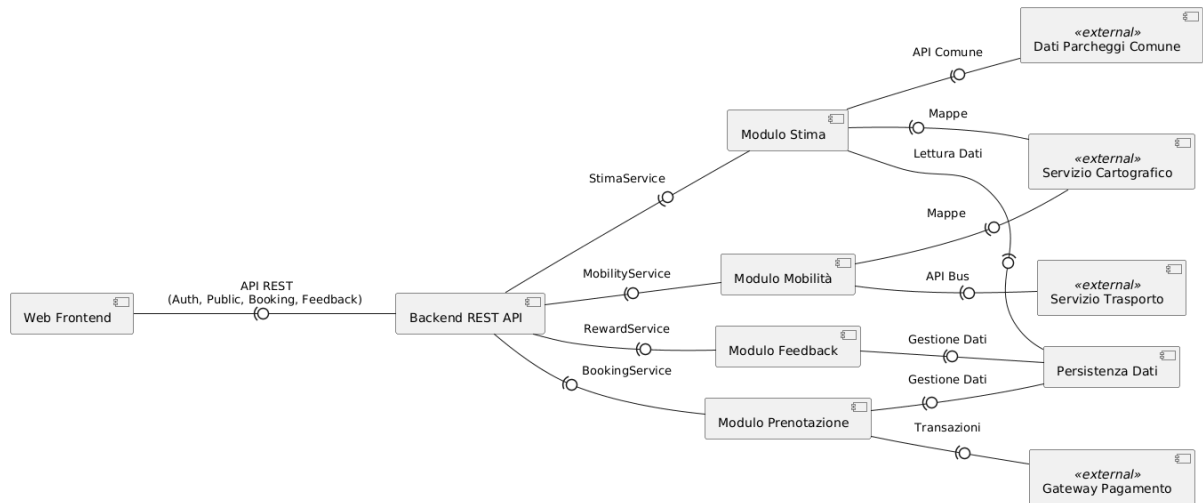


Figura 1: Diagramma dei componenti di TrentoParking

1.2 Descrizione dei componenti

1.2.1 Web Frontend

Il componente Web Frontend rappresenta il punto di accesso principale dell'utente. Permette di visualizzare la mappa, selezionare area e raggio, consultare la stima della disponibilità, ricevere suggerimenti di mobilità alternativa e, per gli utenti autenticati, inviare feedback o prenotare posti privati.

Interfacce fornite

- **UI:** interfaccia grafica accessibile da browser desktop e mobile.

Interfacce richieste

- **API REST:** interfaccia di accesso al backend, che comprende funzionalità di autenticazione, consultazione pubblica, prenotazione e feedback.

1.2.2 Backend REST API

Il Backend REST API espone le funzionalità del sistema tramite endpoint REST. Riceve le richieste dal frontend, valida i parametri in ingresso, coordina i moduli interni e restituisce le risposte in formato JSON. In questa vista architetturale, anche le operazioni amministrative sono veicolate attraverso il backend e non vengono rappresentate come interfaccia separata.

Interfacce fornite

- **API REST:** insieme delle operazioni di autenticazione, consultazione pubblica, gestione feedback e prenotazione.

Interfacce richieste

- **StimaService**

- **MobilityService**
- **RewardService**
- **BookingService**

1.2.3 Modulo Stima

Il Modulo Stima si occupa di elaborare la probabilità di trovare parcheggio gratuito o a pagamento in una determinata area. La stima viene ottenuta combinando dati esterni sui parcheggi, supporto cartografico e dati persistiti utili alla valutazione.

Interfacce fornite

- **StimaService**: servizio per la generazione della stima di disponibilità.

Interfacce richieste

- **API Comune**: accesso ai dati pubblici sui parcheggi del Comune.
- **Mappe**: supporto cartografico per geolocalizzazione e distanze.
- **Lettura Dati**: accesso ai dati persistiti necessari alla stima.

1.2.4 Modulo Mobilità

Il Modulo Mobilità viene attivato quando la disponibilità di parcheggio risulta bassa. Il suo compito è proporre un'alternativa basata sul trasporto pubblico, suggerendo linee e fermate utili a raggiungere la destinazione.

Interfacce fornite

- **MobilityService**: servizio per la generazione di suggerimenti di mobilità alternativa.

Interfacce richieste

- **Mappe**: supporto cartografico.
- **API Bus**: accesso ai dati del trasporto pubblico.

1.2.5 Modulo Feedback

Il Modulo Feedback gestisce l'inserimento, la modifica e la rimozione dei feedback da parte degli utenti autenticati, oltre all'aggiornamento del sistema di premialità.

Interfacce fornite

- **RewardService**: servizio di gestione feedback e premialità.

Interfacce richieste

- **Gestione Dati**: accesso ai dati persistiti relativi a utenti e feedback.

1.2.6 Modulo Prenotazione

Il Modulo Prenotazione rappresenta l'estensione demo innovativa del sistema. Quando la disponibilità di parcheggio è bassa, l'utente autenticato può visualizzare posti privati offerti da host e prenotarli. Il pagamento viene trattato come interazione con un servizio esterno mock.

Interfacce fornite

- **BookingService**: servizio per la gestione della prenotazione di posti privati.

Interfacce richieste

- **Gestione Dati**: accesso a utenti, posti privati e prenotazioni.
- **Transazioni**: interfaccia verso il gateway di pagamento mock.

1.2.7 Persistenza Dati

Il componente Persistenza Dati gestisce l'archiviazione delle informazioni applicative. Contiene dati relativi a utenti, feedback, parcheggi, posti privati, disponibilità orarie e prenotazioni.

Interfacce fornite

- **Lettura Dati**: interfaccia di accesso in lettura.
- **Gestione Dati**: interfaccia di accesso in lettura e scrittura.

Interfacce richieste

- Nessuna.

1.2.8 Dati Parcheggi Comune

Rappresenta il sistema esterno da cui vengono recuperati i dati sui parcheggi pubblici.

Interfacce fornite

- **API Comune**: accesso ai dati parcheggi.

Interfacce richieste

- Nessuna.

1.2.9 Servizio Cartografico

Rappresenta il sistema esterno usato per la visualizzazione geografica e il calcolo delle distanze.

Interfacce fornite

- **Mappe**: interfaccia cartografica.

Interfacce richieste

- Nessuna.

1.2.10 Servizio Trasporto

Rappresenta il sistema esterno che fornisce i dati sul trasporto pubblico.

Interfacce fornite

- **API Bus:** accesso ai dati del trasporto pubblico.

Interfacce richieste

- Nessuna.

1.2.11 Gateway Pagamento

Rappresenta il sistema esterno di pagamento mock utilizzato nella demo della prenotazione privata.

Interfacce fornite

- **Transazioni:** interfaccia per la gestione del pagamento.

Interfacce richieste

- Nessuna.

2 Diagramma delle classi e descrizione delle classi

In questa sezione viene descritta la struttura statica di TrentoParking, attraverso il diagramma delle classi e la descrizione delle classi principali. Il modello comprende le classi necessarie per autenticazione e ruoli, ricerca e stima, prenotazione privata e mobilità alternativa.

2.1 Diagramma delle classi

La Figura 2 riporta il diagramma delle classi del sistema.

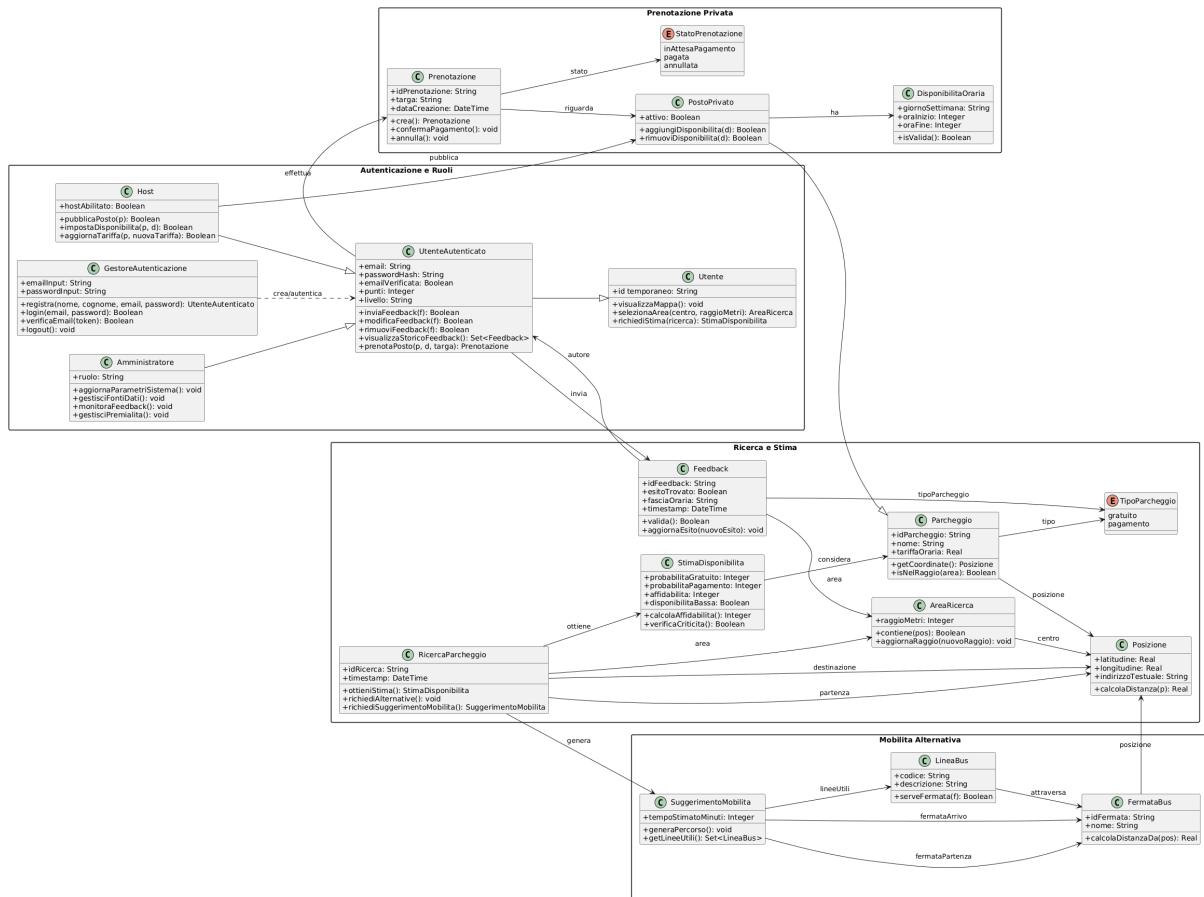


Figura 2: Diagramma delle classi di TrentoParking

2.2 Autenticazione e Ruoli

2.2.1 GestoreAutenticazione

Classe di controllo che gestisce registrazione, login, logout e verifica email.

- **Attributi:** emailInput: String, passwordInput: String
- **Operazioni:**
 - registra(nome, cognome, email, password): UtenteAutenticato
 - login(email, password): Boolean
 - verificaEmail(token): Boolean
 - logout(): void

2.2.2 Utente

Rappresenta l'utente generico del sistema, non necessariamente autenticato.

- **Attributi:** idTemporaneo: String
- **Operazioni:**
 - visualizzaMappa(): void

- selezionaArea(centro, raggioMetri): AreaRicerca
- richiediStima(ricerca): StimaDisponibilita

2.2.3 UtenteAutenticato

Specializzazione di `Utente`. Accede alle funzionalità riservate del sistema.

- **Attributi:** email: String, passwordHash: String, emailVerificata: Boolean, punti: Integer, livello: String
- **Operazioni:**
 - inviaFeedback(f): Boolean
 - modificaFeedback(f): Boolean
 - rimuoviFeedback(f): Boolean
 - visualizzaStoricoFeedback(): Set<Feedback>
 - prenotaPosto(p, d, targa): Prenotazione

2.2.4 Host

Specializzazione di `UtenteAutenticato`. Può offrire un proprio posto auto privato in determinate fasce orarie.

- **Attributi:** hostAbilitato: Boolean
- **Operazioni:**
 - pubblicaPosto(p): Boolean
 - impostaDisponibilita(p, d): Boolean
 - aggiornaTariffa(p, nuovaTariffa): Boolean

2.2.5 Amministratore

Specializzazione di `UtenteAutenticato`. Gestisce parametri e fonti dati del sistema e controlla il meccanismo di premialità.

- **Attributi:** ruolo: String
- **Operazioni:**
 - aggiornaParametriSistema(): void
 - gestisciFontiDati(): void
 - monitoraFeedback(): void
 - gestisciPremialita(): void

2.3 Prenotazione Privata

2.3.1 Prenotazione

Rappresenta la prenotazione di un posto privato.

- **Attributi:** idPrenotazione: String, targa: String, dataCreazione: DateTime
- **Operazioni:**
 - crea(): Prenotazione
 - confermaPagamento(): void
 - annulla(): void

La classe è associata a un posto privato e a uno stato di prenotazione.

2.3.2 PostoPrivato

Specializzazione di Parcheggio. Rappresenta un posto auto privato pubblicato da un host.

- **Attributi:** attivo: Boolean
- **Operazioni:**
 - aggiungiDisponibilita(d): Boolean
 - rimuoviDisponibilita(d): Boolean

La classe è associata a una o più disponibilità orarie.

2.3.3 DisponibilitaOraria

Rappresenta una fascia oraria in cui un posto privato è disponibile.

- **Attributi:** giornoSettimana: String, oraInizio: Integer, oraFine: Integer
- **Operazioni:** isValida(): Boolean

2.3.4 StatoPrenotazione

Enumerazione che rappresenta lo stato di una prenotazione.

- **Valori:** inAttesaPagamento, pagata, annullata

2.4 Ricerca e Stima

2.4.1 RicercaParcheggio

Rappresenta una richiesta di ricerca eseguita dall'utente.

- **Attributi:** idRicerca: String, timestamp: DateTime
- **Operazioni:**
 - ottieniStima(): StimaDisponibilita
 - richiediAlternative(): void

- `richiediSuggerimentoMobilita(): SuggerimentoMobilita`

La classe è associata a un'istanza di `AreaRicerca`, a una posizione di partenza, a una posizione di destinazione e può generare un suggerimento di mobilità.

2.4.2 Feedback

Rappresenta il contributo inviato da un utente autenticato sull'esito della ricerca di parcheggio.

- **Attributi:** `idFeedback: String`, `esitoTrovato: Boolean`, `fasciaOraria: String`, `timestamp: DateTime`
- **Operazioni:**
 - `valida(): Boolean`
 - `aggiornaEsito(nuovoEsito): void`

La classe è associata a un autore di tipo `UtenteAutenticato`, a un'area di ricerca e a una tipologia di parcheggio.

2.4.3 StimaDisponibilita

Contiene il risultato del calcolo effettuato dal sistema.

- **Attributi:** `probabilitaGratuito: Integer`, `probabilitaPagamento: Integer`, `affidabilita: Integer`, `disponibilitaBassa: Boolean`
- **Operazioni:**
 - `calcolaAffidabilita(): Integer`
 - `verificaCriticità(): Boolean`

La classe è associata ai parcheggi considerati per la stima.

2.4.4 AreaRicerca

Rappresenta l'area selezionata dall'utente.

- **Attributi:** `raggioMetri: Integer`
- **Operazioni:**
 - `contiene(pos): Boolean`
 - `aggiornaRaggio(nuovoRaggio): void`

La classe è associata a una posizione centrale.

2.4.5 Parcheggio

Rappresenta un parcheggio rilevante per la ricerca.

- **Attributi:** `idParcheggio: String`, `nome: String`, `tariffaOraria: Real`
- **Operazioni:**

- `getCoordinate(): Posizione`
- `isNelRaggio(area): Boolean`

La classe è associata a una posizione e a una tipologia di parcheggio.

2.4.6 TipoParcheggio

Enumerazione che distingue tra parcheggio gratuito e parcheggio a pagamento.

- **Valori:** gratuito, pagamento

2.4.7 Posizione

Classe di supporto che rappresenta un punto geografico.

- **Attributi:** `latitudine: Real`, `longitudine: Real`, `indirizzoTestuale: String`
- **Operazioni:** `calcolaDistanza(p): Real`

2.5 Mobilità Alternativa

2.5.1 SuggerimentoMobilita

Rappresenta l'alternativa di mobilità proposta dal sistema.

- **Attributi:** `tempoStimatoMinuti: Integer`
- **Operazioni:**
 - `generaPercorso(): void`
 - `getLineeUtili(): Set<LineaBus>`

La classe è associata a una fermata di partenza, una fermata di arrivo e a una o più linee bus utili.

2.5.2 LineaBus

Rappresenta una linea del trasporto pubblico.

- **Attributi:** `codice: String`, `descrizione: String`
- **Operazioni:** `serveFermata(f): Boolean`

La classe è associata alle fermate attraversate.

2.5.3 FermataBus

Rappresenta una fermata del trasporto pubblico.

- **Attributi:** `idFermata: String`, `nome: String`
- **Operazioni:** `calcolaDistanzaDa(pos): Real`

La classe è associata a una posizione geografica.

3 Constraint OCL

In questa sezione vengono riportati i vincoli OCL che completano il diagramma delle classi. I vincoli includono invarianti, precondizioni e postcondizioni.

3.1 Area di ricerca valida

Descrizione: il raggio di ricerca deve essere strettamente positivo.

```
context AreaRicerca
inv raggioPositivo: self.raggioMetri > 0
```

3.2 Coordinate geografiche valide

Descrizione: le coordinate di una posizione devono rientrare negli intervalli geografici ammessi.

```
context Posizione
inv coordinateValide:
    self.latitudine >= -90 and self.latitudine <= 90 and
    self.longitudine >= -180 and self.longitudine <= 180
```

3.3 Tariffa coerente con il tipo di parcheggio

Descrizione: se un parcheggio è gratuito, la tariffa oraria deve essere pari a zero.

```
context Parcheggio
inv tariffaCoerente:
    self.tipo = TipoParcheggio::gratuito implies self.tariffaOraria = 0
```

3.4 Probabilità di stima valide

Descrizione: le probabilità stimate devono essere comprese tra 0 e 100.

```
context StimaDisponibilita
inv probabilitaValide:
    self.probabilitaGratuito >= 0 and self.probabilitaGratuito <= 100 and
    self.probabilitaPagamento >= 0 and self.probabilitaPagamento <= 100
```

3.5 Affidabilità valida

Descrizione: anche il livello di affidabilità della stima deve essere compreso tra 0 e 100.

```
context StimaDisponibilita
inv affidabilitaValida:
    self.affidabilita >= 0 and self.affidabilita <= 100
```

3.6 Feedback completo

Descrizione: ogni feedback deve essere associato a un'area e a una fascia oraria.

```
context Feedback
inv feedbackCompleto:
    self.area <> null and self.fasciaOraria <> ''
```

3.7 Invio feedback consentito solo a utenti verificati

Descrizione: solo un utente autenticato con email verificata può inviare feedback.

```
context UtenteAutenticato::inviaFeedback(f: Feedback): Boolean
pre utenteVerificato:
    self.emailVerificata = true
```

3.8 Aggiornamento punti dopo il feedback

Descrizione: un feedback valido incrementa il punteggio dell'utente.

```
context UtenteAutenticato::inviaFeedback(f: Feedback): Boolean
post puntiAggiornati:
    self.punti = self.punti@pre + 10
```

3.9 Punti non negativi

Descrizione: il punteggio accumulato dall'utente deve essere sempre non negativo.

```
context UtenteAutenticato
inv puntiNonNegativi:
    self.punti >= 0
```

3.10 Fascia oraria valida

Descrizione: una disponibilità oraria deve avere un'ora di inizio precedente all'ora di fine.

```
context DisponibilitaOraria
inv fasciaValida:
    self.oraInizio < self.oraFine
```

3.11 Prenotazione consentita solo a utenti verificati con targa inserita

Descrizione: la prenotazione di un posto privato richiede email verificata e targa non vuota.

```
context UtenteAutenticato::prenotaPosto(p: PostoPrivato, d: DisponibilitaOraria, targa: String): Boolean
pre prenotazioneConsentita:
    self.emailVerificata = true and targa <> ''
```

3.12 Stato iniziale della prenotazione

Descrizione: una nuova prenotazione viene creata nello stato iniziale di attesa pagamento.

```
context UtenteAutenticato::prenotaPosto(p: PostoPrivato, d: DisponibilitaOraria, targ  
post prenotazioneCreata:  
    result.stato = StatoPrenotazione::inAttesaPagamento
```